

CL2002: Data Structure and Algorithms

Lab Project Report

Spring 2026



(Air Traffic Control Simulation System)

Engr. Ayesha Nadeem

Group Members

Name	24I-	Section A/B
Abdullah Muhammad	24I-6560	B
Abdul Hadi	24I-6538	B

Details of Obtained Marks

Question Number	Viva	Working	Report	Total
CLO Number	1	2	3	
Weightage	5	5	5	15
Total Marks	5	5	5	15
Obtained Marks				

Table Of Contents

INTRODUCTION	4
1.1 MOTIVATION.....	4
1.2 PROBLEM STATEMENT	4
DESIGN	5
2.1 REQUIREMENT CONSTRAINTS AND CONSIDERATIONS	5
2.2 FLOWCHART	6
3.1 CODE IMPLEMENTATION	7
3.2 RESULTS.....	21
CONCLUSION	25
4.1 SUMMARY	25
4.2 FUTURE IMPROVEMENTS.....	25
4.3 LESSONS LEARNED.....	25
REFERENCES	26

INTRODUCTION

1.1 MOTIVATION

One of the practical uses of scheduling and prioritization that applies to the real world is air traffic management. Air traffic needs to be scheduled for landing and takeoff in an airport to ensure there are no unnecessary delays. For this reason, we have decided to simulate a simple model of the Air Traffic Control system through Data Structures and Algorithms. We are intrigued by airplanes and aviation in general, thus motivating us to choose this particular project.

1.2 PROBLEM STATEMENT

The objective of the current project is to design an Air Traffic Control (ATC) Simulation System that controls the landing and taking-off process of aircraft in the most efficient way possible. When the aircraft are at the airport ready for landing, the system will prioritize them depending on their proximity to the runway based on Min Heap. The aircraft awaiting takeoff will be served according to the FIFO method using the Queue data structure. This simulation system includes an emergency mechanism where priority will be granted to particular aircrafts for landing.

DESIGN

2.1 REQUIREMENT CONSTRAINTS AND CONSIDERATIONS

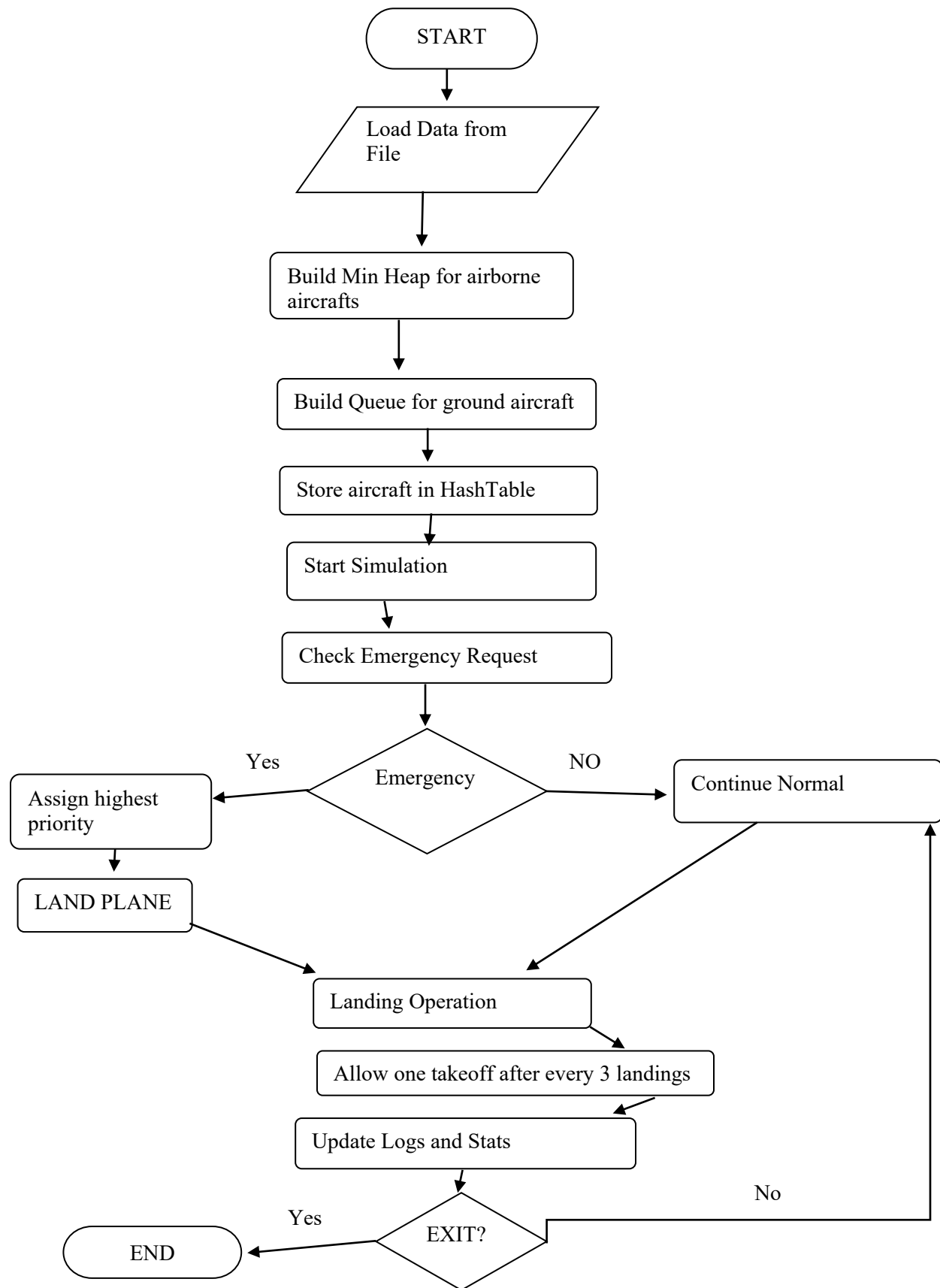
Constraints:

- Maximum heap capacity is fixed at 100 aircraft.
- Landing priority is determined solely by aircraft distance from the runway.
- Emergency aircraft receive the highest priority.
- Takeoff scheduling follows FIFO order.
- One takeoff is allowed after every three landings.
- Flight IDs must be unique.

Considerations:

- Efficient landing management using a Min Heap.
- Fair takeoff scheduling using a Queue.
- Fast flight lookup using a Hash Table.
- Event tracking using a Linked List.
- Separate handling of landing and takeoff operations.
- Simple and modular system design.

2.2 FLOWCHART



IMPLEMENTATION

3.1 CODE IMPLEMENTATION

```
/*  
  
    AIR TRAFFIC CONTROL (ATC) SIMULATION SYSTEM  
    DSA Project  
    CE-4B  
    M.Abdul Hadi (24I-6538)  
    Abdullah Muhammad (24I-6560)  
*/  
  
#include <iostream>  
#include <fstream>  
#include <string>  
#include <cmath>  
#include <ctime>  
using namespace std;  
  
const int HASH_SIZE = 101;  
const int HEAP_CAPACITY = 100;  
const int LANDINGS_BEFORE_TAKEOFF = 3;  
  
struct Plane {  
    string flightID;  
    float distance = 0.0;  
    int priority = 0;  
    bool isEmergency = false;  
    string status = "";  
};  
  
struct MinHeap {  
    Plane* heap[HEAP_CAPACITY];  
    int size;  
  
    MinHeap() { size = 0; }  
  
    int parent(int i) { return (i - 1) / 2; }  
    int leftChild(int i) { return 2 * i + 1; }  
    int rightChild(int i) { return 2 * i + 2; }  
  
    void swapNodes(int i, int j) {  
        Plane* temp = heap[i];  
        heap[i] = heap[j];  
        heap[j] = temp;  
    }  
};
```

```

    heap[j] = temp;
}

void heapifyUp(int i) {
    while (i > 0 && heap[parent(i)]->priority > heap[i]->priority) {
        swapNodes(i, parent(i));
        i = parent(i);
    }
}

void heapifyDown(int i) {
    int smallest = i;
    int l = leftChild(i);
    int r = rightChild(i);
    if (l < size && heap[l]->priority < heap[smallest]->priority)
        smallest = l;
    if (r < size && heap[r]->priority < heap[smallest]->priority)
        smallest = r;
    if (smallest != i) {
        swapNodes(i, smallest);
        heapifyDown(smallest);
    }
}

void insert(Plane* p) {
    if (size >= HEAP_CAPACITY) {
        cout << " [!] Airspace full! Cannot add more planes.\n";
        return;
    }
    heap[size++] = p;
    heapifyUp(size - 1);
}

Plane* extractMin() {
    if (size == 0) return nullptr;
    Plane* top = heap[0];

    size--;
    heap[0] = heap[size];
    heapifyDown(0);
    return top;
}

bool setEmergency(string id) {
    for (int i = 0; i < size; i++) {
        if (heap[i]->flightID == id) {
            heap[i]->priority = 0;
            heap[i]->isEmergency = true;
        }
    }
}

```



```

        heapifyUp(i);
        return true;
    }
}
return false;
}

bool isEmpty() { return size == 0; }

void display() {
    if (size == 0) {
        cout << " (no planes in airspace)\n";
        return;
    }

    cout << "\nFlight\t\tDistance\tPriority\tStatus\n";
    cout << " -----\n";
    for (int i = 0; i < size; i++) {
        cout << " ";
        cout << heap[i]->flightID << "\t\t";
        cout << heap[i]->distance << "\t\t";
        cout << heap[i]->priority << "\t\t";
        if (heap[i]->isEmergency)
            cout << "EMERGENCY\n";
        else
            cout << heap[i]->status << "\n";
    }
}

};

struct QNode {
    Plane* plane;
    QNode* next;
    QNode(Plane* p) : plane(p), next(nullptr) {}
};

struct TakeoffQueue {
    QNode* front;
    QNode* rear;
    int count;

    TakeoffQueue() : front(nullptr), rear(nullptr), count(0) {}

    void enqueue(Plane* p) {
        QNode* node = new QNode(p);
        if (!rear) {

```

```

        front = rear = node;
    }
    else {
        rear->next = node;
        rear = node;
    }
    count++;
}

Plane* dequeue() {
    if (!front) return nullptr;
    QNode* temp = front;
    Plane* p = temp->plane;
    front = front->next;
    if (!front) { rear = nullptr; }
    delete temp;
    count--;
    return p;
}

bool isEmpty() { return front == nullptr; }

void display() {
    if (!front) {
        cout << " no planes waiting for takeoff\n";
        return;
    }
    cout << " The Takeoff order: ";
    QNode* curr = front;
    while (curr) {
        cout << curr->plane->flightID;
        if (curr->next) cout << " -> ";
        curr = curr->next;
    }
    cout << "\n";
}

};

struct HashNode {
    string key;
    Plane* value;
    HashNode* next;
    HashNode(string k, Plane* v) : key(k), value(v), next(nullptr) {}
};

struct HashTable {
    HashNode* table[HASH_SIZE];
};

```

```

HashTable() {
    for (int i = 0; i < HASH_SIZE; i++) table[i] = nullptr;
}

int hashFn(string key) {
    int hash = 0;
    for (int i = 0; i < key.length(); i++) {
        hash += key[i];
    }
    return hash % HASH_SIZE;
}

void insert(Plane* p) {
    int idx = hashFn(p->flightID);
    HashNode* node = new HashNode(p->flightID, p);
    node->next = table[idx];
    table[idx] = node;
}

Plane* lookup(string id) {
    int idx = hashFn(id);
    HashNode* curr = table[idx];
    while (curr) {
        if (curr->key == id) return curr->value;
        curr = curr->next;
    }
    return nullptr;
}

void remove(string id) {
    int idx = hashFn(id);
    HashNode* curr = table[idx];
    HashNode* prev = nullptr;
    while (curr) {
        if (curr->key == id) {
            if (prev) prev->next = curr->next;
            else table[idx] = curr->next;
            delete curr;
            return;
        }
        prev = curr;
        curr = curr->next;
    }
}

};

struct LogNode {
    string timestamp;

```

```

    string event;
    LogNode* next;
    LogNode(string t, string e) : timestamp(t), event(e), next(nullptr) {}
};

struct FlightLog {
    LogNode* head;
    LogNode* tail;
    int eventCount;

    FlightLog() : head(nullptr), tail(nullptr), eventCount(0) {}

    string getTime() {
        time_t now = time(0);
        tm t;
        localtime_s(&t, &now);
        char buf[10];
        sprintf_s(buf, "%02d:%02d:%02d", t.tm_hour, t.tm_min, t.tm_sec);
        return string(buf);
    }

    void addEvent(string event) {
        LogNode* node = new LogNode(getTime(), event);
        if (!tail) {
            head = tail = node;
        }
        else {
            tail->next = node;
            tail = node;
        }
        eventCount++;
    }

    void display() {
        if (!head) {
            cout << " (no events logged yet)\n";
            return;
        }

        cout << "\n Time\t\tEvent\n";
        cout << " -----\n";

        LogNode* curr = head;
        while (curr) {
            cout << " " << curr->timestamp << "\t" << curr->event << "\n";
            curr = curr->next;
        }
    }
};

```

```

        cout << " -----\n";
        cout << " Total events: " << eventCount << "\n";
    }
};

struct ATCSysytem {
    MinHeap    airQueue;
    TakeoffQueue groundQueue;
    HashTable  registry;
    FlightLog  log;

    int landingCount;
    int cycleNumber;
    int totalLandings;
    int totalTakeoffs;
    int totalEmergencies;

    ATCSysytem() {
        landingCount = 0;
        cycleNumber = 0;
        totalLandings = 0;
        totalTakeoffs = 0;
        totalEmergencies = 0;
    }

    void loadFromFile(string filename) {
        ifstream file(filename);
        if (!file.is_open()) {
            cout << "\n [!] Could not open file: " << filename << "\n";

            return;
        }

        string line;
        string section = "";

        while (getline(file, line)) {
            if (line.empty()) continue;

            if (!line.empty() && line.back() == '\r') line.pop_back();

            if (line == "AIR") { section = "AIR";  continue; }
            if (line == "GROUND") { section = "GROUND"; continue; }

            if (section == "AIR") {
                int spacePos = (int)line.find(' ');

```

```

        if (spacePos == (int)string::npos) continue;
        string id = line.substr(0, spacePos);
        float dist = stof(line.substr(spacePos + 1));

        Plane* p = new Plane();
        p->flightID = id;
        p->distance = dist;
        p->priority = (int)(dist * 10);
        p->isEmergency = false;
        p->status = "airborne";

        airQueue.insert(p);
        registry.insert(p);
        log.addEvent("Plane " + id + " entered airspace at " + to_string(dist) + " km");
    }
    else if (section == "GROUND") {
        string id = line;

        Plane* p = new Plane();
        p->flightID = id;
        p->distance = 0;
        p->priority = 999;
        p->isEmergency = false;
        p->status = "waiting";

        groundQueue.enqueue(p);
        registry.insert(p);
        log.addEvent("Plane " + id + " added to takeoff queue");
    }
}

file.close();
cout << " [OK] Data loaded from file: " << filename << "\n";
}

```

```

void addAirPlane() {
    string id;
    float dist;
    cout << "\n Enter Flight ID  : ";
    cin >> id;

    if (registry.lookup(id)) {
        cout << " [!] Flight " << id << " already exists in system.\n";
        return;
    }
}

```

```

    cout << " Enter Distance (km): ";
    cin >> dist;

    Plane* p = new Plane();
    p->flightID = id;

    p->distance = dist;
    p->priority = (int)(dist * 10);
    p->isEmergency = false;
    p->status = "airborne";

    airQueue.insert(p);
    registry.insert(p);

    log.addEvent("NEW: Plane " + id + " entered airspace at " + to_string(dist) + " km");
    cout << " [OK] Plane " << id << " added to airspace.\n";
}

void addGroundPlane() {
    string id;
    cout << "\n Enter Flight ID: ";
    cin >> id;

    if (registry.lookup(id)) {
        cout << " [!] Flight " << id << " already exists in system.\n";
        return;
    }

    Plane* p = new Plane();
    p->flightID = id;
    p->distance = 0;
    p->priority = 999;
    p->isEmergency = false;
    p->status = "waiting";

    groundQueue.enqueue(p);
    registry.insert(p);
    log.addEvent("NEW: Plane " + id + " added to takeoff queue");
    cout << " [OK] Plane " << id << " added to takeoff queue.\n";
}

void runCycle() {
    cycleNumber++;
    cout << "\n CYCLE " << cycleNumber << "\n";
}

```

```

if (landingCount >= LANDINGS_BEFORE_TAKEOFF && !groundQueue.isEmpty()) {
    Plane* p = groundQueue.dequeue();
    p->status = "takeoff";
    cout << " [TAKEOFF] " << p->flightID << " cleared for takeoff after "
        << LANDINGS_BEFORE_TAKEOFF << " landings.\n";
    log.addEvent("TAKEOFF: " + p->flightID + " cleared for takeoff");
    registry.remove(p->flightID);
    landingCount = 0;

    totalTakeoffs++;
    return;
}

if (!airQueue.isEmpty()) {
    Plane* p = airQueue.extractMin();
    p->status = "landed";

    if (p->isEmergency) {
        cout << " [EMERGENCY LANDING] " << p->flightID
            << " landed immediately! (was " << p->distance << " km away)\n";
        log.addEvent("EMERGENCY LAND: " + p->flightID + " landed immediately");

        totalEmergencies++;
    }
    else {
        cout << " [LANDING] " << p->flightID
            << " cleared to land. (distance: " << p->distance << " km)\n";
        log.addEvent("LANDED: " + p->flightID + " landed successfully");
    }

    registry.remove(p->flightID);
    landingCount++;
    totalLandings++;
}
else if (!groundQueue.isEmpty()) {
    Plane* p = groundQueue.dequeue();
    p->status = "takeoff";
    cout << " [TAKEOFF] " << p->flightID << " cleared for takeoff (no air traffic).\n";
    log.addEvent("TAKEOFF: " + p->flightID + " (no air traffic)");
    registry.remove(p->flightID);
    totalTakeoffs++;
}
else {
    cout << " [INFO] No planes in airspace or ground queue. Airport is clear.\n";
    log.addEvent("CYCLE " + to_string(cycleNumber) + ": No operations");
}
}

```



```

void declareEmergency() {
    string id;
    cout << "\n Enter Flight ID to declare emergency: ";
    cin >> id;

    Plane* p = registry.lookup(id);
    if (!p) {
        cout << " [!] Flight " << id << " not found in system.\n";
        return;
    }
    if (p->status != "airborne") {
        cout << " [!] " << id << " is not currently airborne (status: " << p->status << ").\n";
        return;
    }
    if (p->isEmergency) {
        cout << " [!] " << id << " is already declared as emergency.\n";
        return;
    }

    if (airQueue.setEmergency(id)) {
        cout << " [!!] EMERGENCY DECLARED for " << id << "\n";
        cout << " [!!] Priority set to 0 -- will land next cycle.\n";
        log.addEvent("EMERGENCY DECLARED: " + id + " given priority 0");
    }
    else {
        cout << " [!] Could not find " << id << " in airspace heap.\n";
    }
}

void searchPlane() {
    string id;
    cout << "\n Enter Flight ID to search: ";
    cin >> id;

    Plane* p = registry.lookup(id);
    if (!p) {
        cout << " [!] Flight " << id << " not found.\n";
        return;
    }

    cout << "\n Plane Info :\n";
    cout << " Flight ID : " << p->flightID << "\n";
    cout << " Status : " << p->status << "\n";
    cout << " Distance : " << p->distance << " km\n";
    cout << " Priority : " << p->priority << "\n";
}

```

```

        cout << " Emergency : " << (p->isEmergency ? "YES" : "NO") << "\n";

    }

    void viewAirspace() { cout << "\n PLANES IN AIRSPACE :\n";   airQueue.display(); }
    void viewGroundQueue() { cout << "\n TAKEOFF QUEUE (GROUND) \n";
groundQueue.display(); }
    void viewLog() { cout << "\n FLIGHT EVENT LOG \n";      log.display(); }

    void dailyReport() {
        cout << "          DAILY REPORT\n";

        cout << " Total Cycles Run   : " << cycleNumber << "\n";
        cout << " Total Landings     : " << totalLandings << "\n";
        cout << " Total Takeoffs      : " << totalTakeoffs << "\n";
        cout << " Total Emergencies   : " << totalEmergencies << "\n";
        cout << " Planes in Airspace  : " << airQueue.size << "\n";
        cout << " Planes on Ground    : " << groundQueue.count << "\n";
        cout << " Events Logged       : " << log.eventCount << "\n";

    }

    void runMultipleCycles() {
        int n;
        cout << "\n How many cycles to run? ";
        cin >> n;
        for (int i = 0; i < n; i++) {
            if (airQueue.isEmpty() && groundQueue.isEmpty()) {
                cout << "\n [INFO] Airport is clear. No more operations.\n";
                break;
            }
            runCycle();
        }
    }
};

void printBanner() {
    cout << "\n";
    cout << " ----- \n";
    cout << " |   AIR TRAFFIC CONTROL SIMULATION SYSTEM   | \n";
    cout << " ----- \n \n";
}

void printMenu() {
    cout << "\n ----- \n";
    cout << " |           MAIN MENU           | \n";
    cout << " ----- \n";
    cout << " 1. Run One Cycle                \n";
}

```

```

cout << " 2. Run Multiple Cycles          \n";
cout << " 3. Declare Emergency             \n";
cout << " 4. Add New Plane (Airborne)         \n";
cout << " 5. Add New Plane (Ground/Takeoff)     \n";
cout << " 6. View Airspace (Planes in Air)      \n";
cout << " 7. View Takeoff Queue (Ground)        \n";
cout << " 8. Search Plane by Flight ID           \n";
cout << " 9. View Flight Log                    \n";
cout << " 10. Daily Report                     \n";
cout << " 0. Exit                             \n";

cout << " Enter choice: ";
}

int main() {
    printBanner();

    ATCSysSystem atc;

    cout << " Loading airport data...\n";
    atc.loadFromFile("airport_data_test_run.txt");

    cout << "\n [INFO] Takeoff rule: 1 takeoff after every " << LANDINGS_BEFORE_TAKEOFF <<
    " landings.\n";

    int choice;
    do {
        printMenu();
        cin >> choice;

        switch (choice) {
            case 1: atc.runCycle(); break;
            case 2: atc.runMultipleCycles(); break;
            case 3: atc.declareEmergency(); break;
            case 4: atc.addAirPlane(); break;
            case 5: atc.addGroundPlane(); break;
            case 6: atc.viewAirspace(); break;
            case 7: atc.viewGroundQueue(); break;
            case 8: atc.searchPlane(); break;
            case 9: atc.viewLog(); break;
            case 10: atc.dailyReport(); break;
            case 0:
                cout << "\n [ATC] System shutting down. Goodbye!\n\n";
                break;
            default:
                cout << "\n [!] Invalid choice. Try again.\n";
        }
    } while (choice != 0);
}

```

```
    }  
    } while (choice != 0);  
    return 0;  
}
```

3.2 RESULTS

```
Microsoft Visual Studio Debug Console

|-----|
| AIR TRAFFIC CONTROL SIMULATION SYSTEM |
|-----|

Loading airport data...
[OK] Data loaded from file: airport_data_test_run.txt

[INFO] Takeoff rule: 1 takeoff after every 3 landings.

|-----|
| MAIN MENU |
|-----|
1. Run One Cycle
2. Run Multiple Cycles
3. Declare Emergency
4. Add New Plane (Airborne)
5. Add New Plane (Ground/Takeoff)
6. View Airspace (Planes in Air)
7. View Takeoff Queue (Ground)
8. Search Plane by Flight ID
9. View Flight Log
10. Daily Report
0. Exit
Enter choice: 1

CYCLE 1
[LANDING] PA410 cleared to land. (distance: 1.5 km)

|-----|
| MAIN MENU |
|-----|
1. Run One Cycle
2. Run Multiple Cycles
3. Declare Emergency
4. Add New Plane (Airborne)
5. Add New Plane (Ground/Takeoff)
6. View Airspace (Planes in Air)
7. View Takeoff Queue (Ground)
8. Search Plane by Flight ID
9. View Flight Log
10. Daily Report
0. Exit
Enter choice: 2

How many cycles to run? 3

CYCLE 2
[LANDING] QR202 cleared to land. (distance: 3 km)

CYCLE 3
[LANDING] PK301 cleared to land. (distance: 5 km)

CYCLE 4
[TAKEOFF] GT101 cleared for takeoff after 3 landings.
```

```
Microsoft Visual Studio Debug Console
+ v

-----
|          MAIN MENU          |
-----
1. Run One Cycle
2. Run Multiple Cycles
3. Declare Emergency
4. Add New Plane (Airborne)
5. Add New Plane (Ground/Takeoff)
6. View Airspace (Planes in Air)
7. View Takeoff Queue (Ground)
8. Search Plane by Flight ID
9. View Flight Log
10. Daily Report
0. Exit
Enter choice: 3

Enter Flight ID to declare emergency: TK101
[!!] EMERGENCY DECLARED for TK101
[!!] Priority set to 0 -- will land next cycle.

-----
|          MAIN MENU          |
-----
1. Run One Cycle
2. Run Multiple Cycles
3. Declare Emergency
4. Add New Plane (Airborne)
5. Add New Plane (Ground/Takeoff)
6. View Airspace (Planes in Air)
7. View Takeoff Queue (Ground)
8. Search Plane by Flight ID
9. View Flight Log
10. Daily Report
0. Exit
Enter choice: 4

Enter Flight ID : PK747
Enter Distance (km): 40
[OK] Plane PK747 added to airspace.

-----
|          MAIN MENU          |
-----
1. Run One Cycle
2. Run Multiple Cycles
3. Declare Emergency
4. Add New Plane (Airborne)
5. Add New Plane (Ground/Takeoff)
6. View Airspace (Planes in Air)
7. View Takeoff Queue (Ground)
8. Search Plane by Flight ID
9. View Flight Log
10. Daily Report
0. Exit
Enter choice: 5

Enter Flight ID: PK890
[OK] Plane PK890 added to takeoff queue.
```

```
Microsoft Visual Studio Debu | X + v

|-----|
| MAIN MENU |
|-----|
1. Run One Cycle
2. Run Multiple Cycles
3. Declare Emergency
4. Add New Plane (Airborne)
5. Add New Plane (Ground/Takeoff)
6. View Airspace (Planes in Air)
7. View Takeoff Queue (Ground)
8. Search Plane by Flight ID
9. View Flight Log
10. Daily Report
0. Exit
Enter choice: 6

PLANES IN AIRSPACE :

Flight      Distance  Priority  Status
-----
TK101       10        0        EMERGENCY
EK500       8         80       airborne
PK747       40        400      airborne

|-----|
| MAIN MENU |
|-----|
1. Run One Cycle
2. Run Multiple Cycles
3. Declare Emergency
4. Add New Plane (Airborne)
5. Add New Plane (Ground/Takeoff)
6. View Airspace (Planes in Air)
7. View Takeoff Queue (Ground)
8. Search Plane by Flight ID
9. View Flight Log
10. Daily Report
0. Exit
Enter choice: 7

TAKEOFF QUEUE (GROUND)
The Takeoff order: GT102 -> GT103 -> PK890
```

```
Microsoft Visual Studio Debu | X + v

|-----|
| MAIN MENU |
|-----|
1. Run One Cycle
2. Run Multiple Cycles
3. Declare Emergency
4. Add New Plane (Airborne)
5. Add New Plane (Ground/Takeoff)
6. View Airspace (Planes in Air)
7. View Takeoff Queue (Ground)
8. Search Plane by Flight ID
9. View Flight Log
10. Daily Report
0. Exit
Enter choice: 8

Enter Flight ID to search: GT102

Plane Info :
Flight ID : GT102
Status : waiting
Distance : 0 km
Priority : 999
Emergency : NO

|-----|
| MAIN MENU |
|-----|
1. Run One Cycle
2. Run Multiple Cycles
3. Declare Emergency
4. Add New Plane (Airborne)
5. Add New Plane (Ground/Takeoff)
6. View Airspace (Planes in Air)
7. View Takeoff Queue (Ground)
8. Search Plane by Flight ID
9. View Flight Log
10. Daily Report
0. Exit
Enter choice: 9

FLIGHT EVENT LOG

Time      Event
-----
23:11:49  Plane PK301 entered airspace at 5.000000 km
23:11:49  Plane QR202 entered airspace at 3.000000 km
23:11:49  Plane EK500 entered airspace at 8.000000 km
23:11:49  Plane TK101 entered airspace at 10.000000 km
23:11:49  Plane PA410 entered airspace at 1.500000 km
23:11:49  Plane GT101 added to takeoff queue
23:11:49  Plane GT102 added to takeoff queue
23:11:49  Plane GT103 added to takeoff queue
23:11:52  LANDED: PA410 landed successfully
23:11:55  LANDED: QR202 landed successfully
23:11:55  LANDED: PK301 landed successfully
23:11:55  TAKEOFF: GT101 cleared for takeoff
23:12:16  EMERGENCY DECLARED: TK101 given priority 0
23:12:30  NEW: Plane PK747 entered airspace at 40.000000 km
23:12:38  NEW: Plane PK890 added to takeoff queue

Total events: 15
```

```
Microsoft Visual Studio Debug Console
+ -

-----
| MAIN MENU |
-----
1. Run One Cycle
2. Run Multiple Cycles
3. Declare Emergency
4. Add New Plane (Airborne)
5. Add New Plane (Ground/Takeoff)
6. View Airspace (Planes in Air)
7. View Takeoff Queue (Ground)
8. Search Plane by Flight ID
9. View Flight Log
10. Daily Report
0. Exit
Enter choice: 10
    DAILY REPORT
Total Cycles Run : 4
Total Landings : 3
Total Takeoffs : 1
Total Emergencies : 0
Planes in Airspace : 3
Planes on Ground : 3
Events Logged : 15

-----
| MAIN MENU |
-----
1. Run One Cycle
2. Run Multiple Cycles
3. Declare Emergency
4. Add New Plane (Airborne)
5. Add New Plane (Ground/Takeoff)
6. View Airspace (Planes in Air)
7. View Takeoff Queue (Ground)
8. Search Plane by Flight ID
9. View Flight Log
10. Daily Report
0. Exit
Enter choice: 0

[ATC] System shutting down. Goodbye!

C:\LAB 3 DSA\Final\64\Debug\Final.exe (process 3116) exited with code 0 (0x0).
Press any key to close this window . . .
```

3.3 ANALYSIS AND INTREPRETATION OF RESULTS

The results show that the chosen data structures effectively support different ATC scheduling tasks. The Min Heap enables efficient priority-based landing management with $O(\log n)$ insertion and deletion, the Queue ensures fair FIFO takeoff scheduling with $O(1)$ operations, the Hash Table allows rapid flight lookup for emergencies, and the Linked List maintains operational records efficiently. Together, these structures create a realistic and efficient airport traffic management simulation.

CONCLUSION

4.1 SUMMARY

The Air Traffic Control Simulation System was successfully developed using various types of data structures. The landing of the flights was controlled using Min Heap; take off of flights was handled by Queue, information of the flight was handled by Hash Table, and operation records were kept using Linked List. The system ensured efficient scheduling of flights along with emergency handling and reporting.

4.2 FUTURE IMPROVEMENTS

1. Adding real-time aircraft arrival updates.
2. Introducing multiple runways.
3. Developing a graphical user interface.
4. Integrating real-world flight datasets.
5. Supporting concurrent landings and takeoffs.

4.3 LESSONS LEARNED

1. Practical application of Min Heap data structures.
2. Implementation of Queue scheduling systems.
3. Use of Hash Tables for fast searching.
4. Linked List implementation for event logging.
5. Integration of multiple data structures into a single system.
6. Importance of algorithm efficiency in real-world applications.
7. Object-oriented software development in C++.

REFERENCES

- [1] GeeksforGeeks, “Heap Data Structure,” GeeksforGeeks, 2024. [Online]. Available: <https://www.geeksforgeeks.org/heap-data-structure/>. [Accessed: Jun. 6, 2026].
- [2] D. S. Malik, Data Structures Using C++, 2nd ed. Boston, MA, USA: Course Technology, Cengage Learning, 2010.